

# Objectives

- Asynchronous Node.js
- Working with Files

## Asynchronous Node.js

### Introduction

Asynchronous programming is a technique that enables your program to handle multiple tasks *concurrently* without blocking the main execution thread. Instead of waiting for time-consuming operations (like file I/O or HTTP requests) to complete, our code can continue executing other tasks while these operations run in the background.

Node.js is designed around asynchronous, non-blocking I/O, making it ideal for building efficient and scalable server-side applications.

### Promises

A Promise is a built-in JavaScript object that represents the eventual result of an asynchronous operation. It provides a cleaner, more readable alternative to nested callbacks, helping developers avoid the infamous “callback hell.”

A Promise has three states:

| State     | Description                                                                   |
|-----------|-------------------------------------------------------------------------------|
| Pending   | The initial state. The async operation hasn't completed yet.                  |
| Fulfilled | The operation completed successfully. The <code>.then()</code> callback runs. |
| Rejected  | The operation failed. The <code>.catch()</code> callback runs.                |

We create a Promise using the `Promise` constructor, which accepts a function with two parameters:

- `resolve` : Called when the operation succeeds (returns the value)
- `reject` : Called when the operation fails (throws an error)

```
const fs = require('fs');

function readTextFile(path) {
```

```
return new Promise((resolve, reject) => {
  fs.readFile(path, 'utf8', (err, data) => {
    if (err) reject("File read failed");
    else resolve(data);
  });
});

readTextFile("example.txt")
  .then(content => console.log("File content:", content))
  .catch(error => console.log("Error:", error));
```

## The `async` Keyword

When we create an async function, we use the `async` keyword before the function declaration:

```
async function fetchData() {
  // Our async code goes here
}
```

This tells Node.js: We want this function to run asynchronously.

When we call an async function:

- Our program immediately continues to the next line of code
- The async operation runs in the **background**
- This is why we call it **non-blocking**

```
const fs = require('fs').promises;

async function getData() {
  console.log("Starting read...");
  const content = await fs.readFile('data.txt', 'utf8'); // Runs in background
  console.log("File read complete!");
}

getData();
```

```
console.log("We can do other work while waiting!");
```

### Output Order:

1. "Starting read..."
2. "We can do other work while waiting!"
3. "File read complete!"

## The `await` Keyword

The `await` keyword pauses the execution of our `async` function until the specified Promise resolves. This allows us to work with the resolved value directly rather than dealing with the Promise object itself. Importantly, while `await` pauses the current `async` function, it doesn't block the entire program—other parts of our application continue running normally during this waiting period.

There are three crucial aspects to remember about `async/await`:

1. Every `async` function automatically returns a Promise, even if we don't explicitly create one.
2. We can only use the `await` keyword inside functions marked with `async`.
3. While awaiting a Promise, the rest of our program maintains its normal execution flow.

### Example HTTP Requests

We start by downloading the `axios` package `npm install axios`

```
const axios = require('axios');

async function fetchUser() {
  console.log("Fetching user data...");
  const response = await axios.get('https://jsonplaceholder.typicode.com/users/1');
  const user = response.data;
  console.log("User data received:", user);
  return user;
}

fetchUser();
console.log("Loading page content...");
```

We'll see the following output:

```
"Fetching user data..."
"Loading page content..."
"User data received: {...}"
```

## JavaScript Timers

Node.js provides two core timer functions for scheduling code execution:

1. `setTimeout` → Runs a function **once** after a delay
  2. `setInterval` → Repeats a function **continuously** at intervals
- Both are **asynchronous**, meaning they run in the background without blocking other code.

### `setTimeout`

Runs a function **once** after a specified delay

```
setTimeout(() => {
  console.log("This runs after 2 seconds");
}, 2000);
```

### `setInterval`

Repeats a function **continuously** at a fixed interval:

```
let counter = 0;
const timer = setInterval(() => {
  counter++;
  console.log(`Tick ${counter}`);
  if (counter === 8) clearInterval(timer);
}, 1000);
```

## Managing Timers

When working with `setTimeout` and `setInterval`, always:

1. **Store timer IDs in variables:**

```
const timeoutId = setTimeout(...);  
const intervalId = setInterval(...);
```

## 2. Clear them when done:

```
clearTimeout(timeoutId);  
clearInterval(intervalId);
```

# Working with Files in Node.js

## Introductions:

Computers store data in memory while they are running programs and performing calculations. However, this data is temporary. When a program finishes executing, all the variables and results stored in memory are lost.

To prevent this data loss, we can store it in files. This allows us to retain the information even after the program has completed its execution.

## Writing and Appending to Files:

When working with files in **Node.js**, we use the built-in `fs` module (`fs` stands for “file system”).

This module provides both **synchronous** and **asynchronous** methods for file operations. We first import the module:

```
const fs = require('fs');
```

To write data to a file, we can use:

- `fs.writeFileSync(path, data)`  
Writes data to a file **synchronously** (blocks execution until the file is written).
- `fs.writeFile(path, data, callback)`  
Writes data to a file **asynchronously**.
- `fs.appendFile(path, data, callback)`  
Appends new data to the end of an existing file (or creates the file if it doesn't exist).

## Example (Synchronous Write):

```
const fs = require('fs');

const text = "Hello my name is Ali \nnice to meet you All\n";

fs.writeFileSync("output.txt", text);
console.log("File written successfully.");
```

## Example (Asynchronous Append):

```
const fs = require('fs');

const moreText = "This is additional content.\n";

fs.appendFile("output.txt", moreText, (err) => {
  if (err) throw err;
  console.log("Data appended successfully.");
});
```

## Reading Files:

Reading files in Node.js also uses the `fs` module.

We can read files using:

- `fs.readFileSync(path, encoding)`  
Reads the entire file synchronously.
- `fs.readFile(path, encoding, callback)`  
Reads the file asynchronously.
- `fs.createReadStream(path)`  
Used for large files to read in chunks.

## Example (Synchronous Read):

```
const fs = require('fs');

const content = fs.readFileSync("output.txt", "utf8");
```

```
console.log("File content:\n", content);
```

## Example (Asynchronous Read):

```
`const fs = require('fs');
fs.readFile("output.txt", "utf8", (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

## Checking File Existence and Metadata

Before interacting with a file, it is often good to check its status:

```
const fs = require('fs');

const filePath = 'output.txt';
const dirPath = 'my_folder';

// Check if file exists
console.log(fs.existsSync(filePath)); // true if file exists

// Get file size in bytes (if it exists)
if (fs.existsSync(filePath)) {
  const stats = fs.statSync(filePath);
  console.log(stats.size); // file size in bytes
}

// Check if directory exists
console.log(fs.existsSync(dirPath)); // true if directory exists
```

- `fs.existsSync(path)` checks if the file or directory exists.
- `fs.statSync(path).size` gives the size of the file in bytes (stat also provides metadata like modified time, etc.).

## Deleting and Renaming Files

Node.js allows us to rename and delete files easily.

## Renaming a File

```
const fs = require('fs');

fs.rename('old.txt', 'new.txt', (err) => {
  if (err) throw err;
  console.log('File renamed successfully.');
```

- `fs.rename(oldPath, newPath, callback)` changes the file name or moves it to another location.

## Deleting a File

```
const fs = require('fs');

fs.unlink('unwanted.txt', (err) => {
  if (err) throw err;
  console.log('File deleted successfully.');
```

- `fs.unlink(path, callback)` removes the file permanently.

## Task

Using everything we've learned during this course, create a Ruby project that solves a problem you face in your daily life.

Think about tasks you often repeat, information you want to organize, or anything you wish could be automated then build a solution using Node.js!